



Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Universidad Nacional de Rosario

Tecnicatura Universitaria en Inteligencia Artificial

Procesamiento del Lenguaje Natural

Guía de estudio del TP2

Embeddings y búsqueda semántica, desde el TP1

Grupo 7

Civetta, Frank, Fucci y Winter

Material de estudio interno · Octubre de 2026

Índice

Objetivo de Aprendizaje	2
Conceptos Clave (El “Qué” y el “Por qué”)	2
Desarrollo Estructurado	2
1. De dónde venimos: el TP1	2
2. El problema que resuelve el TP2	2
3. Parte A: dos versiones del texto	3
4. Parte B: Word2Vec propio contra SBW	3
5. Parte C: SBERT, un modelo de oración	4
6. Parte D: comparar los tres espacios	4
7. Partes E y F: guardar y buscar en Postgres	4
8. Parte 5: la evaluación	4
9. Parte 6: RAG	6
10. La decisión final: qué modelo usaríamos	6
Ejemplo Práctico / Analogía	6
Autoevaluación Rápida	7
Glosario	7

Objetivo de Aprendizaje

Al terminar vas a poder explicar, con nuestros propios resultados, cómo pasamos del corpus y TF-IDF del TP1 a un buscador semántico con embeddings guardados en Postgres, cómo medimos si es mejor y por qué falla cuando falla.

Conceptos Clave (El “Qué” y el “Por qué”)

- **Corpus:** la colección de textos con la que trabajamos. El nuestro son las 150 sinopsis de libros de la categoría “Clásico” de Lectulandia que juntamos en el TP1. Todo el TP2 se apoya en él.
- **Vector:** una lista de números que representa un texto. Hace falta porque una computadora no compara textos, compara números.
- **TF-IDF:** la representación del TP1. Le da a cada palabra del vocabulario un peso alto si aparece mucho en el texto y poco en el resto del corpus. Es la **línea de base** que el TP2 tiene que superar.
- **Vector disperso y vector denso:** el disperso es largo y casi todo ceros, con una posición por palabra (TF-IDF). El denso es corto, sin ceros, y sus posiciones no corresponden a palabras (embeddings). La diferencia importa porque solo el denso puede acercar textos que dicen lo mismo con otras palabras.
- **Embedding:** vector denso que aprende un modelo para que textos con significado parecido queden cerca. Es la herramienta central del TP.
- **Similitud coseno:** mide el ángulo entre dos vectores: 1 si apuntan igual, 0 si no tienen nada en común. Es la forma en que decidimos qué libro se parece más a una consulta. La **distancia coseno** es 1 menos la similitud.
- **Word2Vec:** modelo que aprende un vector por palabra a partir de las palabras que la rodean. Lo usamos entrenado por nosotros y en su versión pre-entrenada (**SBW**).
- **SBERT:** modelo de oración que devuelve un solo vector para un texto completo, teniendo en cuenta el orden. Fue nuestro mejor modelo.
- **Token y truncado:** los modelos de oración leen pedazos de palabra llamados **tokens**, y SBERT lee como máximo 128. Lo que sigue se descarta sin aviso, y eso explica casi todos nuestros errores.
- **pgvector y HNSW:** pgvector es la extensión de Postgres que guarda vectores y mide distancias. **HNSW** es un índice que acelera la búsqueda de vecinos cercanos a cambio de ser aproximado.
- **precision@k y R-precision:** métricas que cuentan cuántos libros relevantes aparecen arriba del ranking. Son las que nos dicen si un modelo es mejor que otro.
- **RAG:** recuperar documentos con el buscador y pasárselos a un modelo generativo para que responda con ellos. Fue nuestra parte avanzada.

Desarrollo Estructurado

1. De dónde venimos: el TP1

En el TP1 armamos el corpus con un programa en Python (`src/scrapper.py`). **Playwright** abre un navegador sin ventana y recorre el listado de la categoría y la ficha de cada libro. **BeautifulSoup** lee el HTML de cada ficha y saca los datos. El resultado es `data/libros.csv`, con una fila por libro y estos campos: título, autores, géneros, serie, número de serie, sinopsis, URL de la ficha, categoría y fecha de extracción.

Ese corpus se representó con TF-IDF, el último de los métodos de codificación de la Unidad 2 (one-hot, Count Vectorizer, TF-IDF y Hash Vectorizer). La consigna del TP2 remarca la limitación que comparten todos: ninguno captura el significado.

2. El problema que resuelve el TP2

La consigna lo muestra con un ejemplo. Para TF-IDF, “una novela sobre piratas” y “un relato de bucaneros” tienen similitud exactamente 0, porque no comparten ninguna palabra. Un buscador así le falla a cualquiera que no adivine las palabras exactas de la sinopsis.

Los embeddings intentan resolverlo ubicando el significado en un espacio denso, donde la cercanía no depende de que las palabras coincidan. Pero la consigna pone el peso en otra pregunta: **¿cómo sabemos que la solución nueva es mejor que la vieja?** Por eso la evaluación (Parte 5) es el núcleo del TP.

	TF-IDF (TP1)	Embeddings (TP2)
Tipo de vector	Disperso, una posición por palabra	Denso, 100 a 512 posiciones
Qué compara “piratas” contra “bucaneros”	Palabras exactas Similitud 0	Significado aprendido Pueden quedar cerca
Cómo se obtiene	Se calcula contando palabras	Lo aprende un modelo leyendo texto

3. Parte A: dos versiones del texto

Cada limpieza del texto borra información a propósito. Pasar a minúsculas confunde “Madrid” con “madrid”; sacar la puntuación borra dónde termina una oración; sacar las **stopwords** (“el”, “de”, “que”, “no”) borra negaciones y relaciones entre palabras.

No hay una limpieza correcta para todo, así que armamos dos versiones de cada sinopsis:

Versión	Qué tiene	Para qué modelo	Por qué
sinopsis_cruda	El texto original, solo sin espacios repetidos	SBERT	Aprendió sobre oraciones reales y usa el orden y las palabras chicas
sinopsis_limpia	Minúsculas, sin puntuación, números ni stopwords	Word2Vec y TF-IDF	Solo miran qué palabras aparecen

La frase de la consigna que resume esto: el preprocesamiento pertenece al modelo, no al corpus.

4. Parte B: Word2Vec propio contra SBW

Word2Vec parte de una idea: una palabra se conoce por las palabras que la rodean. En la versión **Skip-gram**, el modelo recibe una palabra y tiene que adivinar cuáles aparecen cerca; para acertar ajusta los números del vector de cada palabra, y esos números terminan siendo el embedding.

Entrenamos nuestro propio Word2Vec con las 150 sinopsis limpias:

Parámetro	Valor	Por qué
vector_size	100	Con tan poco texto no hay información para llenar 300 dimensiones
window	5	El contexto son 5 palabras a cada lado
min_count	2	Una palabra que aparece una sola vez no tiene de dónde aprenderse
sg	1	Skip-gram, que anda mejor que CBOW con corpus chicos
seed, workers, hash	42, 1, fijo	Para que cada corrida dé los mismos vectores

SBW (Spanish Billion Words) es un Word2Vec pre-entrenado con muchísimo texto en español, con vectores de 300 dimensiones. El archivo pesa 1,1 GB y no se sube al repositorio.

Al comparar los vecinos de “amor”, “guerra”, “muerte” y “familia”, SBW devuelve palabras con sentido y el nuestro devuelve palabras que solo son frecuentes en el corpus (para “amor”: “vida”, “así”, “obra”).

Para tener un vector por libro, promediamos los vectores de sus palabras. Eso pierde el orden (“el perro mordió al hombre” y “el hombre mordió al perro” dan lo mismo), las negaciones y el peso relativo de cada palabra, y en textos largos los promedios se parecen entre sí. Lo usamos igual porque es simple y representa a la familia “promedio de vectores de palabra” que la consigna pide comparar contra un modelo de oración.

5. Parte C: SBERT, un modelo de oración

SBERT lee el texto entero y devuelve un solo vector. Usamos `distiluse-base-multilingual-cased-v1`, el de la Unidad 2: 512 dimensiones, multilingüe.

Como complemento (no está en la consigna): por dentro es un transformer, una red en la que cada palabra se interpreta mirando a las demás, y por eso el orden y las negaciones cuentan.

Su límite es de **128 tokens**. Contamos los tokens reales de cada sinopsis con el tokenizador del modelo y encontramos que **134 de las 150 se truncan**. La consigna advierte que el modelo no avisa: hay que ir a buscarlo.

6. Parte D: comparar los tres espacios

1. **Ranking lado a lado** con 3 consultas, para ver a ojo si los resultados tienen sentido.
2. **Similitud entre pares al azar** (300 pares). Un buen espacio separa: dos libros cualquiera deberían parecerse poco. En nuestro Word2Vec la media es **0,999**: todo se parece a todo y el espacio no discrimina, aunque igual devuelva un ranking.
3. **Proyección PCA a 2 dimensiones**, coloreada por género, con la **varianza explicada**: Word2Vec propio 96,1 % (señal de que está colapsado en pocas direcciones) y SBERT 11,5 % (el dibujo muestra muy poco del espacio real, así que una nube mezclada puede estar separada en las 512 dimensiones).

7. Partes E y F: guardar y buscar en Postgres

La base es un Postgres en Supabase con **pgvector**, que agrega el tipo `vector(n)` y el operador `<=>` (distancia coseno).

1. **Una tabla por modelo**: `embeddings_word2vec_propio` (100) y `embeddings_sbert` (512). No pueden compartir columna porque `vector(n)` tiene dimensión fija y el índice compara vectores de un solo largo. Son dos familias distintas, como pide la consigna.
2. **Normalización**: cada vector se divide por su largo para que mida 1. Antes de insertar se descartan los nulos o no finitos (no hubo ninguno).
3. **Upsert**: si el libro ya está, se actualiza la fila en lugar de duplicarla.
4. **Índice HNSW** con la opclass `vector_cosine_ops`, la que corresponde al operador `<=>`. Si no coinciden, Postgres no usa el índice.
5. **Búsqueda**: `buscar(consulta, k, filtro_genero)` calcula el vector de la consulta con SBERT y Postgres ordena por distancia y aplica el filtro con `WHERE`.

El problema de recall con filtro. HNSW revisa unos pocos candidatos (`ef_search`, 40 por defecto) y el `WHERE` se aplica sobre ellos. Si el género es raro (solo 10 libros de Aventuras), pueden quedar menos resultados de los pedidos. Soluciones: subir `ef_search`, usar `hnsw.iterative_scan` de `pgvector` 0.8, índices parciales por género o búsqueda exacta sobre el grupo filtrado.

Las credenciales van en un `.env` que git ignora. `.env.example` es la plantilla sin secretos que muestra qué variables hacen falta.

8. Parte 5: la evaluación

El conjunto de evaluación (`queries.json`): 12 consultas escritas antes de correr los modelos, cada una con sus libros relevantes (entre 2 y 7, contando todas las ediciones).

Tipo	Consultas	Qué prueba
Léxicas	q01 a q03	Comparten palabras con los relevantes; terreno de TF-IDF
Semánticas	q04 a q08	Dicen lo mismo con otras palabras
Sin palabras en común	q09 a q12	Ni una palabra ni raíz compartida; TF-IDF da 0 por construcción

Las métricas. precision@k es la fracción de relevantes entre los primeros k resultados. Su máximo depende de cuántos relevantes tenga la consulta: con 2 relevantes, precision@5 no pasa de 0,4. **R-precision** mira los primeros R resultados, con R igual a la cantidad de relevantes, y su máximo siempre es 1.

Las líneas de base. TF-IDF sobre la misma sinopsis limpia, y el azar: valor esperado = relevantes / 150 (0,025 en promedio) y, para ver cuánto puede dar la suerte, 10.000 simulaciones con **percentil 95 = 0,067**.

Los empates. En TF-IDF muchas similitudes valen 0. Desempatamos con un número al azar fijo por libro (semilla 42), igual para todos los métodos, para que nadie acierte por el orden del CSV.

Método	P@5	P@10	R-prec.	R-prec. léxicas	R-prec. semánticas	R-prec. sin palabras comunes
Azar (esperado)	0,025	0,025	0,025	0,029	0,025	0,022
TF-IDF	0,283	0,217	0,357	0,746	0,410	0,000
Word2Vec propio	0,167	0,125	0,159	0,270	0,129	0,112
SBW	0,267	0,208	0,286	0,571	0,202	0,175
SBERT	0,350	0,258	0,466	0,857	0,414	0,238

¿La diferencia es real? Consulta por consulta, SBERT le gana a TF-IDF en 5, empata en 5 y pierde en 2. El **test de signos** pregunta qué tan raro sería eso si los dos fueran igual de buenos (como tirar una moneda en cada consulta): $p \approx 0,45$. Con 12 consultas no se puede afirmar que SBERT sea mejor; sí se ve dónde se separan (sin palabras en común: 0 contra 0,24). Para los otros: Word2Vec propio gana 2, empata 4 y pierde 6 ($p = 0,29$); SBW gana 3, empata 5 y pierde 4 ($p = 1,0$).

Control SQL contra numpy. El top-5 de Postgres coincidió con el de numpy en todas las consultas menos q01 (4 de 5). Con índice, con `ef_search = 200` y sin índice el resultado fue idéntico, así que el índice no perdía nada: había un empate exacto (distancia 0,6651) entre las dos ediciones de *Las mil y una noches*, iguales en sus primeros 128 tokens.

El caso de falla. Consulta: “obra de teatro sobre un gobernante que pierde la razón o acaba atormentado por sus actos” (relevantes: las dos ediciones de *Macbeth* y *El rey Lear*). SBERT devolvió *Preparación del actor*, *Julio César*, *La vida es sueño*, *Hamlet* y una sola *Macbeth* en el quinto puesto. La hipótesis fue el truncado, y la verificamos con el tokenizador:

Libro	Frase clave	Token donde empieza	¿La lee SBERT?
<i>Macbeth</i>	“tormento” / “remordimientos”	367 / 383	No
<i>El rey Lear</i>	“perder la razón” / “locura”	292 / 316	No
<i>Macbeth. Una tragedia en 5 actos</i>	“mal que nace del ansia de poder”	45	Sí (es la que encontró)

Qué mide y qué no mide la métrica. Cuenta relevantes según nuestro criterio. No mide el orden dentro del top-k, ni la relevancia parcial (*Germinal* para “lucha de clases” cuenta como

error), ni la utilidad para un usuario real. Los juicios son nuestros (sesgo de anotador) y son solo 12 consultas.

El Word2Vec propio, en resumen: colapsado (0,999), inestable (sin semilla sacó 0,067 en P@5, con semilla 42 saca 0,167) y sin vocabulario para las consultas difíciles. 150 sinopsis no alcanzan para entrenar embeddings.

9. Parte 6: RAG

1. **Recuperar:** SBERT trae los 4 libros más parecidos a la pregunta.
2. **Armar el contexto:** cada libro va con una etiqueta [D1] a [D4], su título, autores y sinopsis completa.
3. **Generar:** el contexto y la pregunta van a un modelo de la API de NVIDIA NIM con temperature=0. NVIDIA retiró los Llama durante el TP, así que el código elige el primer modelo disponible de una lista de preferencia; usó google/gemma-4-31b-it.
4. **Evaluar:** chequeos automáticos (citas, citas inválidas, títulos no recuperados, abstención) y revisión manual de las 18 respuestas.

Prompt	Qué pide
v1, ingenuo	Responder usando los libros
v2, estricto	Solo el contexto, citar [Dn] en cada dato, frase fija de abstención, señalar premisas falsas, no nombrar libros externos

Las 9 preguntas: 4 respondibles, 2 sin respuesta en el catálogo, 2 con **premisa falsa** (Holmes en la Luna, Robinson Crusoe con su esposa) y 1 sobre el error del corpus (*Middlemarch*, cuya sinopsis es en realidad un fragmento del *Lazarillo de Tormes*).

Resultados: no hubo **alucinaciones**. Las dos versiones respondieron bien cuando se recuperó el libro y corrigieron las premisas falsas; la ventaja del v2 fue un formato controlable. Todas las fallas fueron de recuperación: *Morfina* quedó en el puesto 12 porque “morfina” aparece en el token 289, y las preguntas que nombran autor o título (Camus, *Middlemarch*) no traen esos libros porque solo indexamos la sinopsis. Además, ser fiel al contexto no es decir la verdad: el v1 dijo que “ninguno de los libros del catálogo” trataba la morfina cuando solo había visto 4.

10. La decisión final: qué modelo usaríamos

SBERT. Mejor promedio y no queda por debajo de TF-IDF en ningún tipo de consulta; corre en una computadora común, sin costo por consulta, las consultas no salen de nuestro servidor y es reproducible. Solo depende de Hugging Face para la descarga inicial. Su debilidad es el truncado, que resolveríamos partiendo las sinopsis en fragmentos (*chunking*), con TF-IDF como apoyo para nombres propios.

Ejemplo Práctico / Analogía

Pensá en una biblioteca con dos bibliotecarios.

El primero (TF-IDF) tiene un fichero ordenado por palabras. Si le pedís “un relato de bucaneros”, busca la ficha “bucaneros”, y si ninguna sinopsis usa esa palabra exacta, te dice que no hay nada, aunque el estante esté lleno de libros de piratas.

El segundo (embeddings) acomodó los libros en un gran salón según de qué tratan: los de piratas en un rincón, los de guerra en otro, los de amor trágico cerca de los de muerte. Cuando le pedís algo, ubica tu pedido en el salón y te trae los libros que están más cerca. Por eso “piratas” y “bucaneros” caen en el mismo rincón. La similitud coseno es medir esa cercanía, y el índice HNSW es un plano con atajos para no recorrer el salón entero.

Ahora el detalle que explica nuestros errores: el segundo bibliotecario, para decidir dónde va cada libro, lee solo las primeras líneas de la contratapa (los 128 tokens). Si lo que hace a *El rey*

Lear un libro sobre la locura está en el último párrafo, lo pone en el rincón de “teatro y reyes” y nunca en el de “gobernantes que pierden la razón”.

Dónde falla la analogía: el salón real no tiene 3 dimensiones sino 512, y sus direcciones no son temas con nombre (“piratas”, “guerra”). Nadie decide qué significa cada número; el modelo lo aprende solo.

Autoevaluación Rápida

1. ¿Por qué para TF-IDF “una novela sobre piratas” y “un relato de bucaneros” tienen similitud 0, y qué cambia con un embedding?
2. ¿Por qué SBERT recibe la sinopsis cruda y Word2Vec la limpia? ¿Qué pasaría si le diéramos la limpia a SBERT?
3. Si SBERT supera a TF-IDF en el promedio, ¿por qué no podemos afirmar que es mejor? ¿Qué nos dicen el test de signos y el tipo de consulta?
4. ¿Cuál fue el caso de falla de SBERT y cómo comprobamos que la causa era el límite de 128 tokens?
5. ¿Qué problema aparece al combinar el índice HNSW con un filtro por género, y qué harías para resolverlo?

Glosario

Término	Qué significa
Corpus	Colección de textos con la que se trabaja; el nuestro son 150 sinopsis
Token	Pedazo de texto que lee un modelo: una palabra o parte de una
Stopwords	Palabras muy frecuentes que casi no dicen nada del tema (“el”, “de”, “que”)
Vector disperso	Largo y casi todo ceros, una posición por palabra (TF-IDF)
Vector denso	Corto y con valores en todas las posiciones (embeddings)
Embedding	Vector denso aprendido para representar el significado
Similitud coseno	Cercanía según el ángulo entre dos vectores; 1 es idéntico
Distancia coseno	1 menos la similitud; es lo que calcula el operador $\leq$
Normalizar	Dividir un vector por su largo para que mida 1
Skip-gram	Entrenar Word2Vec prediciendo las palabras vecinas de cada palabra
Truncado	Corte del texto que supera el límite de tokens del modelo
PCA	Reduce muchas dimensiones a pocas conservando la mayor variación
Varianza explicada	Parte de la información original que conserva la PCA
pgvector	Extensión de Postgres para guardar vectores y buscar por distancia
HNSW	Índice aproximado que acelera la búsqueda de vecinos cercanos
ef_search	Cantidad de candidatos que revisa HNSW en cada búsqueda
Opclass	Opción del índice que fija la distancia (vector_cosine_ops)
Upsert	Insertar una fila o actualizarla si ya existe
Recall	Proporción de resultados buenos que el sistema encuentra
Relevante	Libro que, según nuestro criterio, debería aparecer para una consulta

Término	Qué significa
precision@k	Fracción de relevantes entre los primeros k resultados
R-precision	precision@R, con R igual a la cantidad de relevantes
Línea de base	Método simple contra el que se compara (TF-IDF y el azar)
Percentil 95	Valor que solo se supera en el 5 % de los casos
Test de signos	Prueba si ganar más consultas que perder puede ser casualidad
p-valor	Qué tan común sería el resultado por azar; alto significa “puede ser suerte”
RAG	Recuperar documentos y pasárselos a un modelo generativo
Alucinación	Dato inventado por el modelo que no está en sus fuentes
Prompt	Instrucciones que se le dan al modelo generativo
Premisa falsa	Pregunta que da por cierto algo que no lo es
Chunking	Partir textos largos en fragmentos para no perder el final